

# torch.amin#

<https://pytorch.org/docs/stable/generated/torch.amin.html#torch.amin>

## Description:

Returns the minimum value of each slice of the input tensor in the given dimension(s) dim. `amax/amin` supports reducing on multiple dimensions, `amax/amin` does not return indices. Both `amax/amin` evenly distribute gradients between equal values when there are multiple input elements with the same minimum or maximum value.

## Function Signature

---

```
torch.amin(input, dim, keepdim=False, *, out=None) → Tensor#
```

The difference between `max/min` and `amax/amin` is:

For `max/min`:

Parameters

Keyword Arguments

## Parameters

---

### Keyword

out (Tensor, optional) – the output tensor.

## Code Examples

---

```
>>> a = torch.randn(4, 4)
>>> a
tensor([[ 0.6451, -0.4866,  0.2987, -1.3312],
        [-0.5744,  1.2980,  1.8397, -0.2713],
        [ 0.9128,  0.9214, -1.7268, -0.2995],
        [ 0.9023,  0.4853,  0.9075, -1.6165]])
>>> torch.amin(a, 1)
tensor([-1.3312, -0.5744, -1.7268, -1.6165])
```

## Notes & Warnings

---

### NOTE

Note The difference between max/min and amax/amin is: amax/amin supports reducing on multiple dimensions, amax/amin does not return indices. Both amax/amin evenly distribute gradients between equal values when there are multiple input elements with the same minimum or maximum value. For max/min: If reduce over all dimensions(no dim specified), gradients evenly distribute between equally max/min values. If reduce over one specified axis, only propagate to the indexed element.

## Detailed Documentation

---

### Documentation

Returns the minimum value of each slice of the input tensor in the given dimension(s) dim.

amax/amin supports reducing on multiple dimensions,

amax/amin does not return indices.

Both amax/amin evenly distribute gradients between equal values when there are multiple input elements with the same minimum or maximum value.

If reduce over all dimensions(no dim specified), gradients evenly distribute between equally max/min values.

If reduce over one specified axis, only propagate to the indexed element.

If `keepdim` is `True`, the output tensor is of the same size as input except in the dimension(s) `dim` where it is of size 1. Otherwise, `dim` is squeezed (see `torch.squeeze()`), resulting in the output tensor having 1 (or `len(dim)`) fewer dimension(s).

`input` (Tensor) – the input tensor.

`dim` (int or tuple of ints, optional) – the dimension or dimensions to reduce. If `None`, all dimensions are reduced.

`keepdim` (bool, optional) – whether the output tensor has `dim` retained or not. Default: `False`.

`out` (Tensor, optional) – the output tensor.